

# RepuestosAlToque

Stack tecnologico - Plan e estimacion para pasar de mockup a app real

Bariloche, AR | Documento B - Stack | 29/05/2026

Objetivo: convertir el prototipo navegable actual (HTML/CSS/JS, datos simulados) en una aplicacion funcional con datos reales, manteniendo identica la experiencia visual. Este documento resume la recomendacion de stack, alternativas y un cronograma estimado.

## 1. Por que Supabase (lo mas simple y barato)

Para un MVP chico, 'simple y barato' significa escribir el menor backend posible y operar la menor infraestructura posible. El caso necesita 5 cosas: login con roles, cotizaciones en vivo (websockets), guardar fotos, pagos (MercadoPago) y datos relacionales. Supabase resuelve 4 de esas 5 de forma manejada, sin que tengamos que construirlas ni mantenerlas:

| Necesidad            | Con Supabase                   | Si lo armamos nosotros (Rails/Java) |
|----------------------|--------------------------------|-------------------------------------|
| Base de datos        | Postgres incluido              | Hostear Postgres aparte             |
| Login + roles        | Auth + RLS incluido            | Programarlo (Devise/JWT)            |
| Cotizaciones en vivo | Realtime (websockets) incluido | Servidor de websockets a mantener   |
| Fotos                | Storage incluido               | S3/R2 + integracion                 |
| Pagos                | Va aparte igual                | Va aparte igual                     |

Costo Supabase: se arranca en \$0 (free tier: 500MB DB, 1GB storage, 50k usuarios). Recien se paga ~US\$25/mes al crecer. Vercel es gratis para esto. Total para arrancar: ~\$0/mes (solo dominio y la comision por transaccion de MercadoPago).

## 2. Ruby o Java (tu conocimiento de backend)

### - Java / Spring:

lo descartaria para este MVP. Es el mas pesado y lento de levantar para algo tan chico. Se justifica en sistemas grandes, no aca.

### - Ruby on Rails:

alternativa legitima y tambien barata, sobre todo si vos vas a mantenerlo. Da realtime (ActionCable), fotos (ActiveStorage) y con Hotwire/Turbo hasta sirve las vistas. Monolito simple, ~US\$15/mes. Costo: mas dias de desarrollo, pero jugas con tu fortaleza y tenes control total.

**Recomendacion: arrancar con Next.js + Supabase. Ahorra ~30-40% del trabajo de backend justo en lo que el caso necesita (realtime, auth, storage). Reservar Ruby para cuando haya logica de negocio pesada del lado del servidor que valga la pena ser dueño.**

### 3. Plan + cronograma estimado

Estimado en dias-dev (un desarrollador enfocado). Cada fase queda deployada y demostrable. La columna 'Extra Rails' es lo que sumaria hacerlo en Ruby en lugar de Supabase.

| # | Fase                      | Que queda andando                                | Dias   | Extra Rails |
|---|---------------------------|--------------------------------------------------|--------|-------------|
| 0 | Setup & fundaciones       | Repo Next, Supabase, design system, deploy       | 1      | +0,5        |
| 1 | Schema + RLS + seed       | Base con catalogo de autos y comercios           | 1,5-2  | +0,5        |
| 2 | Auth + roles + onboarding | Login/registro por rol con guardas               | 2-3    | +2          |
| 3 | Flujo de pedido (real)    | Mecanico crea pedido + sube foto (Storage)       | 2      | +1          |
| 4 | Cotizaciones en Realtime  | Comercio cotiza / mecanico ve en vivo + 10 min   | 3-4    | +2          |
| 5 | Anonimato + MercadoPago   | Checkout, webhook, revelado post-pago + comision | 3-4    | +1          |
| 6 | Entrega + reputacion      | Comprobante de entrega + ratings + tiers         | 2-3    | +1          |
| 7 | Admin con datos reales    | Metricas, aprobaciones, reset de contrasenas     | 2-3    | +0,5        |
| 8 | Pulido + QA mobile        | Estados error/vacio, RLS, listo para lanzar      | 2-3    | +0,5        |
|   | TOTAL                     |                                                  | ~19-25 | +8-9        |

### Traduccion a calendario

- **Full-time (~5 dias/sem):**  
~4-5 semanas (Supabase) / ~6-7 semanas (Rails).
- **Part-time (~3 dias/sem):**  
~6-8 semanas (Supabase) / ~9-11 semanas (Rails).

### Hitos demostrables

- **Fin de Fase 4:**  
el corazon funciona con datos reales (pedido -> cotizacion en vivo). Mejor momento para probar con 2-3 comercios reales.
- **Fin de Fase 5:**  
se cobra de verdad con MercadoPago.
- **Fin de Fase 8:**  
listo para piloto en Bariloche.

### Nota sobre MercadoPago (afecta Fase 5)

- **Cobro centralizado:**  
la plataforma cobra todo y le paga al comercio aparte. Simple; es lo estimado. Recomendado para el piloto.
- **Marketplace / split:**  
MP reparte automatico y descuenta comision, pero cada comercio debe conectar su cuenta MP (OAuth) + aprobacion de MP. Suma ~2-3 dias.

## Decisiones pendientes antes de implementar

- 1) Next.js + Supabase (recomendado) o Rails (tu fortaleza).
- 2) Ritmo full-time o part-time (para fijar fecha objetivo).
- 3) MercadoPago centralizado (recomendado para piloto) o split desde el arranque.